

Influur Take-Home Assignment

Creator Intake Review Agent

Senior Full-Stack Engineering Exercise | Next.js, Node.js, PostgreSQL, Render, Tailwind CSS, AI Agent

Confidentiality Notice

This document and the assignment details are confidential and proprietary to Influur. They are provided only for the candidate evaluation process. Do not copy, publish, post, discuss, distribute, or share this information outside Influur or the authorized interview process without written permission from Influur.

1. Assignment Overview

Build a small internal tool called **Creator Intake Review Agent**. The app should help a campaign manager review creator applications for a brand campaign, run an AI-powered fit review, and save decisions.

Expected time	4-5 hours for a senior engineer
AI tools	Allowed. Candidates may use Cursor, Claude, ChatGPT, Copilot, or any other development assistant.
Primary focus	UI quality, full-stack execution, PostgreSQL persistence, backend API design, AI workflow design, deployment, and engineering judgment.
Important constraint	This should be a workflow product, not a generic chatbot.

2. Required Stack

Frontend	Next.js
Backend	Next.js Route Handlers / API routes running on Node.js
Database	PostgreSQL
Hosting	Render
Styling	Tailwind CSS
Primary color	#ff5a00
AI	OpenAI, Anthropic, or an equivalent LLM API

3. Product Context

At Influur, campaign managers need to review creator applications, understand whether a creator is a good fit for a campaign, identify risks, and decide the next action. The goal is to build a practical internal tool that supports this workflow end to end.

4. Core Requirements

4.1 Hosted App on Render

- The app must be deployed on Render and accessible through a live public URL.
- The submitted Render URL must be usable without local setup.

- The README must explain deployment and environment variable setup.

4.2 PostgreSQL Persistence

- Use PostgreSQL for persistence.
- Store campaigns, creator applications, AI review results, agent run history, status changes, and manual notes.
- Refreshing the page must preserve AI results, statuses, and manual notes.

4.3 Next.js + Node.js Backend

- Use Next.js backend routes for server-side logic.
- LLM API keys must never be exposed to the frontend.
- Separate UI logic, API logic, database access, and AI/prompt logic where reasonable.

4.4 AI Agent Workflow

- The AI agent should evaluate a creator application against the campaign context.
- The agent must return structured output and save its result to PostgreSQL.
- Each agent run must create an audit/history record, including success or failure.
- The agent should avoid hallucinating facts not present in the campaign or creator data.

5. UI Requirements

UI Emphasis

We will place strong emphasis on UI/UX quality. The app should not look like a raw technical demo. It should feel like a clean internal tool that a campaign manager could realistically use.

- Use Tailwind CSS for styling.
- Use #ff5a00 as the primary brand color for primary actions and accents.
- Prioritize clarity, usability, and responsiveness over flashy effects.
- Avoid excessive animations. Small hover states, transitions, loading states, and tasteful micro-interactions are fine.
- The UI must be responsive across desktop, tablet, and mobile widths.

5.1 Dashboard View

- Show all creator applications in a clean table, card layout, or responsive combination of both.
- Show creator name, handle, platform, followers, engagement rate, status, latest fit score, and latest AI recommendation.
- Include filters for status and platform.
- Include sorting by followers and fit score.
- Allow the user to open a creator detail page or modal.
- Use clear status badges and a visually scannable layout.

5.2 Creator Detail View

- Show creator profile data, campaign context, application message, latest AI recommendation, reasoning, risks, missing information, suggested reply, manual notes, and agent run history.
- Provide clear actions: Run AI Review, Approve, Reject, Mark as Needs Info, and Save Manual Note.
- Make the primary action visually clear using #ff5a00.
- Organize the page into readable sections so the user never feels lost.

5.3 UI Quality Bar

- Clean spacing, typography, and hierarchy.
- Accessible contrast and readable text.
- Clear loading, empty, and error states.

- No cluttered dashboards or hidden core actions.
- A campaign manager should be able to understand the workflow without explanation.

6. Suggested Data Model

Candidates may adapt this model, but the application should include these core concepts.

```
Campaign {
  id
  brandName
  campaignName
  campaignGoal
  targetAudience
  platforms
  budgetRange
  requirements
  brandSafetyNotes
  createdAt
}

CreatorApplication {
  id
  campaignId
  creatorName
  handle
  platform
  followers
  engagementRate
  audienceSummary
  contentStyle
  applicationMessage
  pastBrandDeals
  status // pending, approved, rejected, needs_info
  manualNote
  createdAt
  updatedAt
}

AgentReview {
  id
  applicationId
  fitScore
  recommendation // approve, reject, needs_info, manual_review
  reasoning
  risks
  missingInfo
  suggestedReply
  createdAt
}

AgentRun {
  id
  applicationId
  status // success, failed
  rawInput
  rawOutput
  errorMessage
  createdAt
}
```

7. Required Backend Capabilities

Campaigns	Fetch campaign details used for creator review.
Applications	Fetch all applications, fetch one application, filter/sort where appropriate.
Status updates	Approve, reject, mark needs info, and persist status changes.
Manual notes	Save and retrieve manual notes for a creator application.
AI review	Run AI review server-side, parse structured output, save AgentReview.

Run history

Save and retrieve AgentRun records for auditability and debugging.

Example endpoints may include `GET /api/applications`, `GET /api/applications/:id`, `PATCH /api/applications/:id`, `POST /api/applications/:id/review`, and `GET /api/applications/:id/runs`. Endpoint names are flexible.

8. AI Agent Requirements

The agent should take campaign data and creator application data as input and produce a structured review. The output should be machine-readable and persisted in PostgreSQL.

```
{
  "fitScore": 8,
  "recommendation": "approve",
  "reasoning": "The creator's audience, content style, and past beauty partnerships strongly match the campaign.",
  "risks": [
    "High follower count may require higher budget",
    "Need to confirm usage rights"
  ],
  "missingInfo": [
    "Exact deliverable pricing",
    "Timeline availability"
  ],
  "suggestedReply": "Hi Mia, thanks for applying! Your content feels like a strong fit for GlowPop..."
}
```

- Use campaign data and creator application data as grounding context.
- Return fitScore, recommendation, reasoning, risks, missingInfo, and suggestedReply.
- Save successful results to AgentReview.
- Save every run attempt to AgentRun, including failed runs.
- Handle malformed or failed AI responses gracefully in the UI.

9. Seed Data Requirement

- Seed at least 1 campaign and 6-8 creator applications.
- Seed data should be realistic enough to test different recommendations: strong fit, weak fit, needs info, and manual review.
- Do not use real confidential creator, brand, or customer data.

Example Campaign

```
{
  "brandName": "GlowPop",
  "campaignName": "GlowPop Summer Launch",
  "campaignGoal": "Drive awareness and trial for a new SPF lip balm among Gen Z consumers.",
  "targetAudience": "Gen Z women, beauty lovers, college students, skincare beginners",
  "platforms": ["TikTok", "Instagram"],
  "budgetRange": "$2,000 - $10,000 per creator",
  "requirements": [
    "Creator must be comfortable showing product usage on camera",
    "Must mention SPF benefits",
    "Must not make medical claims",
    "Content should feel fun, casual, and summer-focused"
  ],
  "brandSafetyNotes": "Avoid creators with controversial content, heavy profanity, or unsafe skincare claims."
}
```

Example Creator Applications

```
[
  {
    "creatorName": "Mia Lopez",
    "handle": "@miaglow",
    "platform": "TikTok",
```

```

    "followers": 820000,
    "engagementRate": 4.9,
    "audienceSummary": "Gen Z women interested in beauty, skincare, and affordable routines.",
    "contentStyle": "Fast-paced GRWM videos, humor, product reactions.",
    "applicationMessage": "I can create a summer GRWM using the lip balm before heading to the beach.",
    "pastBrandDeals": ["e.l.f.", "Bubble Skincare"],
    "status": "pending"
  },
  {
    "creatorName": "Daniel Kim",
    "handle": "@danielreviews",
    "platform": "YouTube",
    "followers": 410000,
    "engagementRate": 6.2,
    "audienceSummary": "Tech and productivity audience, mostly men 18-34.",
    "contentStyle": "Detailed review videos and comparison content.",
    "applicationMessage": "I can review the product from an everyday carry perspective.",
    "pastBrandDeals": ["Notion", "Samsung"],
    "status": "pending"
  },
  {
    "creatorName": "Ava Martinez",
    "handle": "@avastyle",
    "platform": "Instagram",
    "followers": 1200000,
    "engagementRate": 2.7,
    "audienceSummary": "Fashion, travel, luxury lifestyle audience.",
    "contentStyle": "Polished aspirational photo and Reel content.",
    "applicationMessage": "This could fit naturally into my summer travel content.",
    "pastBrandDeals": ["Revolve", "Dior Beauty"],
    "status": "pending"
  }
]

```

10. Submission Requirements

1. Live Render app URL.
2. GitHub repository URL.
3. README with setup instructions.
4. PostgreSQL schema, migrations, or equivalent database setup instructions.
5. Seed instructions.
6. Environment variable instructions.
7. Short explanation of the AI agent design.
8. Known tradeoffs and what the candidate would improve with more time.

11. Evaluation Rubric

Area	Weight	What We Look For
UI / Product Quality	35%	Polished Tailwind UI, intuitive dashboard, responsive layout, clear actions, good information hierarchy, strong loading/empty/error states, tasteful use of #ff5a00, usable by a non-technical campaign manager.
Backend + PostgreSQL	25%	Clean Next.js/Node backend routes, sensible schema, real persistence, saved status changes, saved manual notes, saved agent reviews, saved run history, no exposed API keys.
AI Agent Design	25%	Structured output, clear grounding in campaign and creator data, persisted results, audit trail, failure handling, prompt boundaries, no hallucinated creator facts.
Deployment + Engineering Judgment	15%	Working Render deployment, clear README, local setup clarity, documented environment variables, reasonable tradeoffs, readable maintainable code.

12. Hard Requirements / Dealbreakers

- Must be deployed on Render.
- Must use PostgreSQL.
- Must use Next.js for the frontend.
- Must use Next.js/Node.js backend routes for server-side logic.
- Must use Tailwind CSS for styling.
- Must use #ff5a00 as the primary brand color.
- Must persist data after page refresh.
- Must save AI review results and agent run history.
- Must not expose the LLM API key in frontend code.
- Must include README and setup instructions.

13. Red Flags

- Generic chatbot instead of an operational workflow tool.
- Unstyled or hard-to-use UI.
- No responsive behavior.
- Local-only data with no PostgreSQL persistence.
- AI result not saved to the database.
- No agent run history or auditability.
- Malformed AI response crashes the app.
- Broken Render deployment or missing setup instructions.
- API keys exposed in client-side code.

Final Confidentiality Reminder

This assignment is confidential. The candidate may use the assignment only for Influur interview evaluation. The candidate must not share, publish, post, or distribute the prompt, data, implementation details, or any derivative material outside the authorized interview process.